

MIVA

OPEN UNIVERSITY

COS 201

Computer Programming I – Laboratory Outline

Miva Open University

Headquarters

Plot 1059, O.P. Fingesi Road,

Utako, Abuja

Course Development Team

Instructional Designer: Abdullahi Abubakar Kawu

Language Editor: Udochi Obiukwu, MA English Language

Content Editor: Prof Sylvanus Ehikioya

Table of Contents

COS 201: Computer Programming I – Laboratory Outline 3

Learning Outcomes 4

LAB 1: Introduction to C Programming and Development Environment Setup 8

Experiment 1.1 : Install and configure a C development environment (JetBrains Clion). 9

Exercise 1.1: Students are to Setup their PCs with the JetBrains CLion IDE. 14

Experiment 1.2: Write and execute a simple "Hello, World!" program. 14

Exercise 1.2: Modify the program to print a personalized greeting 16

Experiment 1.3: Create a program that calculates and displays the sum and average of two numbers entered by the user. 16

Exercise 1.3: Create a program that calculates and displays the sum, average and standard deviation of five numbers entered by the user. 17

LAB 2: Variables, Data Types, and Operators 18

Experiment 2.1: C Programming: Declaring and Initializing Variables 19

Exercise 2.1: Write a program to convert temperature from Fahrenheit to Celsius 23

Experiment 2.2: Perform arithmetic and relational operations. 24

Exercise 2.2: Calculate and display area of a rectangle with user-provided dimensions. 28

LAB 3: Control Structures and Conditional Statements 29

Experiment 3.1: Implement conditional statements (if, else if, else) 30

Exercise 3.1: Create a program to determine whether a given number is even or odd 35

Experiment 3.2: Use logical operators for complex conditions. 35

Exercise 3.2: Develop a calculator program that performs basic operations (addition, subtraction, multiplication, division) based on user input. 40

LAB 4: Loops and Iteration 41

Experiment 4.1: Implement for loops, while loops and do-while loops for repetitive tasks 42

Exercise 4.1: Write a program to find the factorial of a user-provided integer using a

loop 47

Experiment 4.2: Utilize loop control statements (break and continue). 47

Exercise 4.2: Implement a guessing game where the user has to guess a randomly generated number within a specified range. 51

LAB 5: Arrays and Strings 52

Experiment 5.1: A simple C program that demonstrates how to declare, initialize, and work with a one-dimensional array. 53

Exercise 5.1: Create a program to find the sum and average of elements in an array accepting input from the user. 55

Exercise 5.2: Implement a program that finds the largest and smallest values in an array accepting input from the user and traverses the array. 60

LAB 6: Functions and Modular Programming 61

Exercise 6.1: Develop a program that calculates the area of various geometric shapes (circle, rectangle, triangle) using separate functions 64

Experiment 6.2: Understand function prototypes and header files. 64

Exercise 6.2: Create a library of utility functions (e.g., factorial, power) and use them in multiple programs. 66

LAB 7: Pointers and Dynamic Memory Allocation 67

Experiment 7.1: Write a C program to create, initialize, assign and access a pointer variable 67

Exercise 7.1: Write a C program to Swap the values of two variables using pointers. 69

Experiment 7.2: Write a C program to create memory for int, char and float variable at run time (using malloc and free) 69

Exercise 7.2: Implement a program that dynamically allocates memory for an array based on user input and displays the elements in reverse order. 72

LAB 8: File Handling and Structures 73

Experiment 8.1: Perform basic file input and output operations 73

Exercise 8.1: Create a program that reads data from a given text file, processes it to store the values in the text file into an array, find the average of the number and writes the results to another file. 75

Exercise 8.2: Design a structure to represent a student's information (name, roll number, marks) and perform operations on a list of students 78

COS 201: Computer Programming I – Laboratory Outline

Learning Outcomes

At the end of this course, students should be able to

1. identify different programming paradigms and their approaches to programming,
2. write programmes in C using basic data types and strings,
3. design and implement programming problems using selection,
4. design and implement programming problems using loops and use & implement classes as data abstractions in an object-oriented approach.
5. implement simple exception handling in programmes, develop programmes with input/output from text files and design and implement programming problems involving arrays.

Lab work: Programming assignments involving hands-on practice in the design and implementation of simple algorithms such as finding the average, and standard deviation, searching, and sorting. Practice developing and tracing simple recursive algorithms. Developing programmes involving inheritance and polymorphism.

Lab 1: Introduction to C Programming and Development Environment Setup

- Experiment 1.1: Install and configure a C development environment (JetBrains Clion).
- Experiment 1.2: Write and execute a simple "Hello, World!" program.
- Experiment 1.3: Create a program that calculates and displays the sum and average of two numbers entered by the user.
- Exercise 1.1: Students to Create a New Project Named **MyFirstCProgram,** and Run.
- Exercise 1.2: Modify the program to print a personalized greeting.
- Exercise 1.3: Create a program that calculates and displays the sum, average and standard deviation of five numbers entered by the user.

Lab 2: Variables, Data Types, and Operators

- Experiment 2.1: Declare and initialize variables of different data types.
- Experiment 2.2: Perform arithmetic and relational operations.
- Exercise 2.1: Write a program to convert temperature from Fahrenheit to Celsius.
- Exercise 2.2: Calculate and display the area of a rectangle with user-provided dimensions.

Lab 3: Control Structures and Conditional Statements

- Experiment 3.1: Implement conditional statements (if, else if, if else ladder).
- Experiment 3.2: Use logical operators for complex conditions.
- Exercise 3.1: Create a program to determine whether a given number is even or odd.

- Exercise 3.2: Develop a calculator program that performs basic operations (addition, subtraction, multiplication, division) based on user input.

Lab 4: Loops and Iteration

- Experiment 4.1: Implement while, do while and for loops for repetitive tasks.
- Experiment 4.2: Utilize loop control statements (break and continue).
- Exercise 4.1: Write a program to find the factorial of a user-provided integer using a loop.
- Exercise 4.2: Implement a guessing game where the user has to guess a randomly generated number within a specified range.

Lab 5: Arrays and Strings

- Experiment 5.1: Work with one-dimensional arrays, including array declaration and initialization.
- Experiment 5.2: Perform array manipulation and traversal.
- Exercise 5.1: Create a program to find the sum and average of elements in an array.
- Exercise 5.2: Implement a program that finds the largest and smallest values in an array accepting input from the user and traverses the array..

Lab 6: Functions and Modular Programming

- Experiment 6.1: Write a simple C program that defines a function to add two numbers
- Experiment 6.2: Understand function prototypes and header files.

- Exercise 6.1: Develop a program that calculates the area of various geometric shapes (circle, rectangle, triangle) using separate functions.
- Exercise 6.2: Create a library of utility functions (e.g., factorial, power) and use them in multiple programs.

Lab 7: Pointers and Dynamic Memory Allocation

- Experiment 7.1: Write a C program to create, initialize, assign and access a pointer variable
- Experiment 7.2: Write a C program to create memory for int, char and float variable at run time(using malloc and free).
- Exercise 7.1: Write a C program to swap the values of two variables using pointers.
- Exercise 7.2: Implement a program that dynamically allocates memory for an array based on user input and displays the elements in reverse order.

Lab 8: File Handling and Structures

- Experiment 8.1: Perform basic file input and output operations.
- Experiment 8.2: Define and use structures to organize data.
- Exercise 8.1: Create a program that reads data from a given text file, processes it to store the values in the text file into an array, find the average of the number and writes the results to another file.
- Exercise 8.2: Design a structure to represent a student's information (name, roll number, marks) and perform operations on a list of students.

Lab 9: Final Project (CA) - 2 weeks

Programming With C - Lab

LAB 1: Introduction to C Programming and Development Environment Setup

Objectives:

- To be familiar with the Programming Environment
- To have a good understanding of the syntax and structure of C programming
- Typing and Executing a C Program

Experiment 1.1 : Install and configure a C development environment (JetBrains Clion).

To efficiently run any software, you should understand the software and hardware requirements. There are different types of Integrated Development Environment (IDE) software used for programming. For the purpose of this Course, we will be using JetBrains Clion which supports the GNU C/C++ Compiler. However, others exist such as GCC / Turbo C/C++. The GNU C/C++ compiler is the most popular and freely available compiler.

System requirements for JetBrains Clion:

- 64-bit version of Windows 10 1809 and later, or Windows Server 2019 and later
- 2 GB free RAM minimum, 8 GB of total system RAM recommended
- 3.5 GB hard disk space, SSD recommended
- 1024x768 minimum screen resolution

1. Visit https://www.jetbrains.com/clion
2. Click on **Download button** on the Menu to Visit the relevant Installer based on your Operating System (OS) or visit https://www.jetbrains.com/clion/download and select your OS
3. Click on Download and wait for the download to complete
4. After Download is completed, follow the on screen instructions to install the IDE on your PC/machine. If you're using Windows, you should see a screen similar to Figure 1 below. Click on Next till you finish the installation.

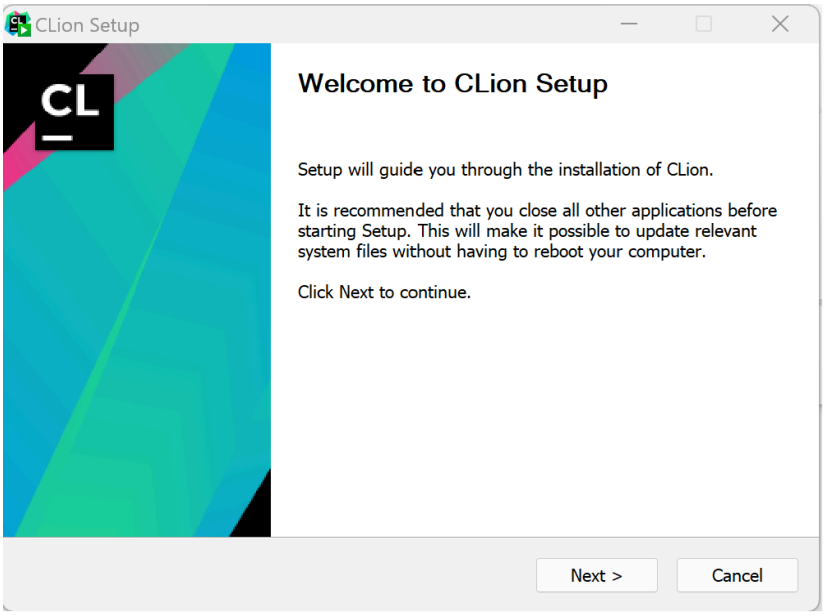


Figure 1: Clion Setup For Windows

5. After the installation is completed, you should see a screen similar to this Figure 2 below:

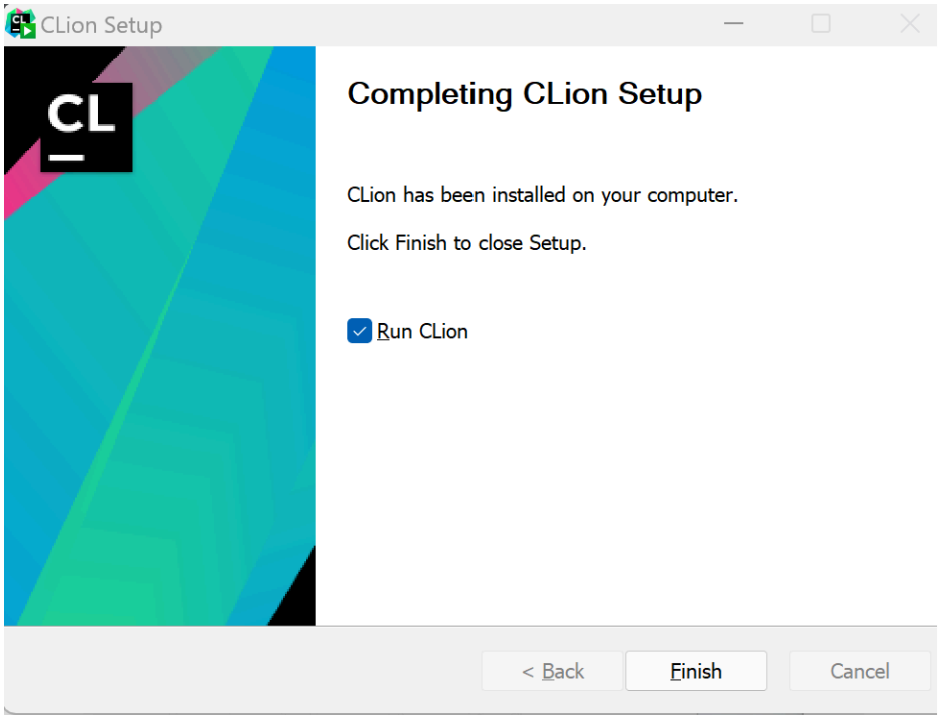


Figure 2: Clion Setup Completed

6. **Launch and Configure CLion:**

- Once installed, launch CLion.
- During the first startup, CLion will ask you to configure your development environment settings (like UI theme, etc.)
- Set up your toolchain (like MinGW or other suitable compiler and debugger). For the purpose of this Lab, we will install/configure the MinGW toolchain.

Note, you can find alternative installations for other toolchain here: https://www.jetbrains.com/help/clion/quick-tutorial-on-configuring-clion-on-windows.html=

Post-Installation

- Licensing
- After installation, you will need to activate CLion. As a Miva Student, you will be given a licence to activate the product with a JetBrains Account.
- Toolchains and Compilers:

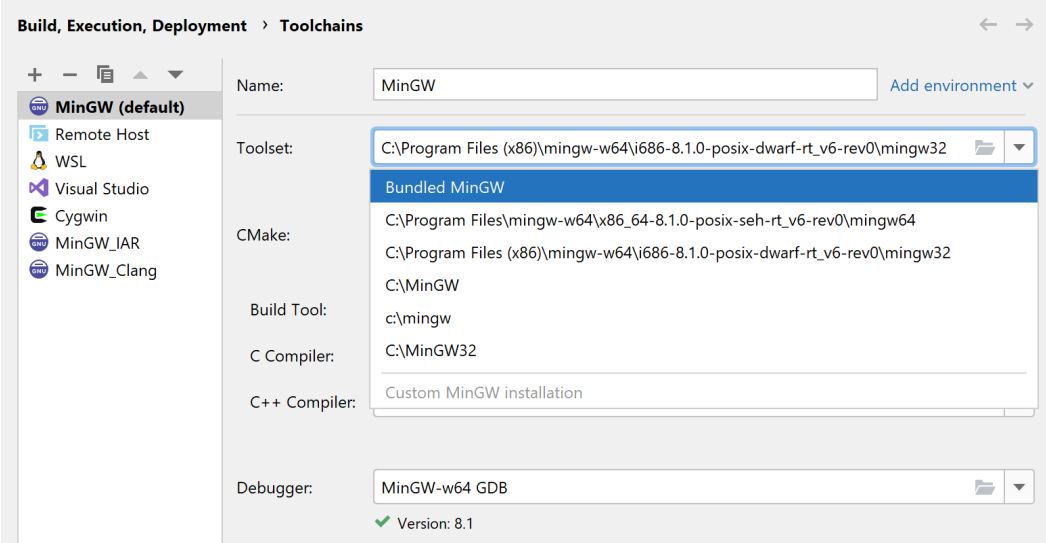
Note: CLion does not come with a compiler or debugger - you will need to install these separately and configure them in CLion. Instructions for doing this are available in the CLion documentation. Also, always refer to the [official CLion documentation](https://www.jetbrains.com/clion/documentation/documentation.html) for the most detailed and accurate instructions.

7. **Configure a MinGW toolchain**

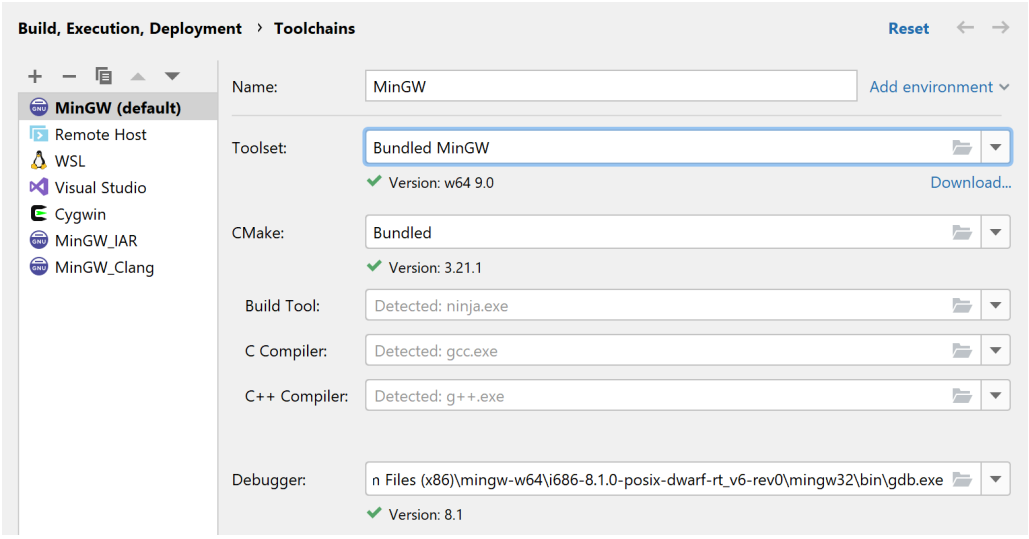
Go to File | Settings | Build, Execution, Deployment | Toolchains.

Click + and select MinGW to add a new MinGW toolchain.

In the Toolset field, you will see Bundled MinGW, which is the default option. If required, open the field to select from the list of other available installations:



Wait until the tool's detection finishes.



Select the Debugger: you can use either bundled GDB, your MinGW GDB, or a custom GDB binary. Bundled GDB is recommended, since it is guaranteed to include Python support required for CLion data renderers. GDB does not yet

support debugging on Windows ARM64. For this type of system, CLion's bundled GDB is intended to be used for remote debugging only. If required, specify the script to initialize the environment.

Click Apply when all the tools are set correctly.

When using a custom MinGW installation, if CLion cannot detect the compilers, double-check the installed packages in MinGW Installation Manager.

Exercise 1.1: Students are to Setup their PCs with the JetBrains CLion IDE.

Instructions:

- As demonstrated in the experiment, students complete their installations and configuration.

Activity 1.1: Installation and Configuration.

Experiment 1.2: Write and execute a simple "Hello, World!" program.

Algorithm: PrintHelloWorld

1. Start
2. Include the necessary header file: #include <stdio.h>
3. Declare the main function: int main()
4. Inside the main function: a. Print the string "Hello, World!" followed by a newline: printf("Hello, World! \n");
5. Return 0 to indicate successful execution: return 0;
6. End

Solution:

```c
#include <stdio.h>

int main() {
    /* my first program in C */
    printf("Hello, World! \n");

    return 0;
}
```

Output:

```
Hello, World!
```

Note:

- The first line of the program **#include <stdio.h>** is a preprocessor command, which tells a C compiler to include stdio.h file before going to actual compilation.
- The next line **int main()** is the main function where the program execution begins.
- The next line **/*...*/** will be ignored by the compiler and it has been put to add additional comments in the program. So such lines are called comments in the program.
- The next line **printf(...)** is another function available in C which causes the message "Hello, World!" to be displayed on the screen. The **\n** in the **printf(...)** stands for line break, which breaks anything after, to a new line.

- The next line **return 0**; terminates the main() function and returns the value 0.

Exercise 1.2: Modify the program to print a personalized greeting

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.

Experiment 1.3: Create a program that calculates and displays the sum and average of two numbers entered by the user.

Algorithm: Calculate Sum And Average

1. Start
2. Declare variables: a, b, sum, average
3. Display "Enter any two integers:"
4. Read values for a and b from the user using scanf
5. Calculate the sum of a and b: sum = a + b
6. Calculate the average of the sum: average = sum / 2
7. Display "Sum and average of two integers: sum average"
8. Display the values of sum and average using printf
9. End`

Code Solution:

```c
#include <stdio.h>

int main() {
    int a, b;
    int sum, average;

    printf("Enter any two integers: ");
    scanf("%d %d", &a, &b);

    sum = a + b;
    average = sum / 2;

    printf("Sum and average of two integers: %d %d", sum, average);

    return 0;
}
```

Output

For example, if the user enters 5, and 8, the output would be:

```
Enter any two integers: 5 8
Sum and average of two integers: 13 6
```

Exercise 1.3: Create a program that calculates and displays the sum, average and standard deviation of five numbers entered by the user.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show

  their purposes)

LAB 2: Variables, Data Types, and Operators

Objectives:

- Understand the concept of variables and their importance in programming.
- Learn how to declare variables in C.
- Get introduced to various data types available in C.
- Learn to initialize variables during declaration.

Experiment 2.1: C Programming: Declaring and Initializing Variables

- Write a simple c program to demonstrate declaring and initializing an integer number.

Algorithm 2.1.1:

1. Include Header File: Begin with #include <stdio.h> for input/output operations.
2. Declare main Function: Start with int main() as the entry point of the program.
3. Declare and Initialize Integer: Use int to declare an integer variable, assigning it a name (e.g., age) and an initial value.
4. Print the Value: Display the integer's value using printf with a format specifier like %d.
5. Return from main: End main with return 0; for successful execution.
6. End main Function: Close the main function with a closing curly brace (}).

Code Solution:

c Copy code

```c
#include <stdio.h>

int main() {
    int age = 25; // Declaring and initializing an integer variable

    printf("Age: %d\n", age); // Printing the value of the integer variable

    return 0;
}
```

Output:



- **Write a simple c program to demonstrate declaring and initializing a floating number.**

Algorithm 2.1.2: Float (Floating Point)

1. Include Header File: Begin with #include <stdio.h> for input/output operations.
2. Declare main Function: Start with int main() as the entry point of the program.
3. Declare and Initialize float: Use int to declare a float variable, assigning it a name (e.g., height) and an initial value.
4. Print the Value: Display the float's value using printf with a format specifier like %.2f.
5. Return from main: End main with return 0; for successful execution.
6. End main Function: Close the main function with a closing curly brace (}).

Code Solution:

```c
#include <stdio.h>

int main() {
    float height = 5.8; // Declaring and initializing a float variable

    printf("Height: %.2f meters\n", height); // Printing the value of the fl

    return 0;
}
```

Output:

```
Height: 5.80 meters
```

- Write a simple c program to demonstrate declaring and initializing a character variable.

Algorithm 2.1.3: Char (Character)

1. Include Header File: Begin with #include <stdio.h> for input/output operations.
2. Declare main Function: Start with int main() as the entry point of the program.
3. Declare and Initialize char (Character): Use char to declare a character variable, assigning it a name (e.g., initial) and an initial value.
4. Print the Value: Display the char's value using printf with a format specifier like %c.
5. Return from main: End main with return 0; for successful execution.

6. End main Function: Close the main function with a closing curly brace (}).

Code Solution:

```c
#include <stdio.h>

int main() {
    char initial = 'A'; // Declaring and initializing a character variable

    printf("Initial: %c\n", initial); // Printing the value of the character

    return 0;
}
```

Output:

```makefile
Initial: A
```

- Write a simple c program to demonstrate declaring and initializing a double-floating number.

Algorithm 2.1.4

double (Double floating-point)

1. Include Header File:Begin with #include ‹stdio.h› for input/output operations.
2. Declare main Function: Start with int main() as the entry point of the program.
3. Declare and Initialize Double:Use double to declare a double variable, assigning it a name (e.g., balance) and an initial value.

4. Print the Value:Display the double's value using printf with a format specifier like %.4f".
5. Return from main:End main with return 0; for successful execution.
6. End main Function:Close the main function with a closing curly brace (}).

Code Solution:

```c
#include <stdio.h>

int main() {
    double balance = 12345.6789; // Declaring and initializing a double vari

    printf("Balance: %.4lf\n", balance); // Printing the value of the double

    return 0;
}
```

Output:

```makefile
Balance: 12345.6789
```

Exercise 2.1: Write a program to convert temperature from Fahrenheit to Celsius

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

Write a comment to make your programs readable.

Use descriptive variables in your programs(Name of the variables should show their purposes)

Activity 2.1: Write a program to convert temperature from Fahrenheit to Celsius

Experiment 2.2: Perform arithmetic and relational operations.

Arithmetic Operations

- Write a simple C program to demonstrate Arithmetic operations.

Algorithm:

1. Start
2. Declare four variables: num1, num2 (to store user input), and sum, difference, product (to store results of operations). Additionally, declare a float variable quotient to store the result of division.
3. Prompt the user to input two numbers and use scanf to read them.
4. Calculate the sum, difference, and product of num1 and num2.
5. Check whether num2 is not equal to zero to avoid division by zero:
    - If true, calculate the quotient and print all operation results.
    - If false, print all results except for the quotient and display an error message regarding division by zero.
6. End

Code:

```c
#include <stdio.h>

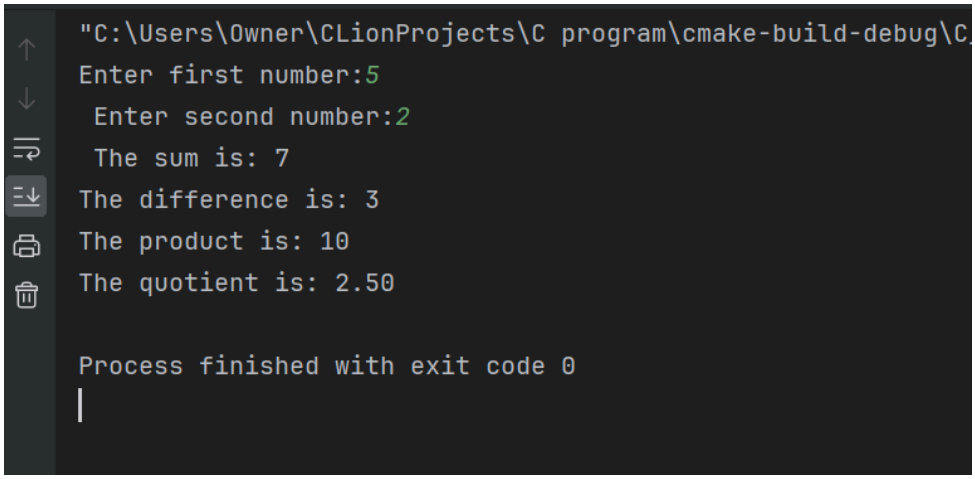
int main() {
    int num1, num2;
    int sum, difference, product;
    float quotient;

    // Get input from the user
    printf( format: "Enter first number: ");
    scanf( format: "%d", &num1);
    printf( format: "Enter second number: ");
    scanf( format: "%d", &num2);
```

```c
    // Perform arithmetic operations
    sum = num1 + num2;
    difference = num1 - num2;
    product = num1 * num2;

    // Check for division by zero
    if(num2 != 0) {
        quotient = (float)num1 / num2;
        printf( format: "The sum is: %d\n", sum);
        printf( format: "The difference is: %d\n", difference);
        printf( format: "The product is: %d\n", product);
        printf( format: "The quotient is: %.2f\n", quotient);
    } else {
        printf( format: "The sum is: %d\n", sum);
        printf( format: "The difference is: %d\n", difference);
        printf( format: "The product is: %d\n", product);
        printf( format: "Cannot divide by zero.\n");
    }
    return 0;
}
```

Output:



Relational Operations

- Write a simple C program to demonstrate Relational operation.

Algorithm:

1. Start
2. Declare two integer variables a and b.
3. Prompt the user to input two integers and store them in a and b respectively.
4. Perform the relational operations and print the results as follows:
    - Compare if a is equal to b and display the result.
    - Compare if a is not equal to b and display the result.
    - Check if a is greater than b and display the result.
    - Check if a is less than b and display the result.
    - Check if a is greater than or equal to b and display the result.
    - Check if a is less than or equal to b and display the result.
5. End

Code:

```c
#include <stdio.h>

int main() {
    int a, b;

    // Get input from the user
    printf("Enter first number: ");
    scanf("%d", &a);

    printf("Enter second number: ");
    scanf("%d", &b);

    // Demonstrate relational operations
    printf("a == b: %d\n", a == b); // Equal to
    printf("a != b: %d\n", a != b); // Not equal to
    printf("a > b: %d\n", a > b);   // Greater than
    printf("a < b: %d\n", a < b);   // Less than
    printf("a >= b: %d\n", a >= b); // Greater than or equal to
    printf("a <= b: %d\n", a <= b); // Less than or equal to

    return 0;
}
```

Output: example if the user enter 7 and 4

```
"C:\Users\Owner\CLionProjects\C program\cmake-bui
Enter first number:7
 Enter second number:4
 a == b: 0
a != b: 1
a > b: 1
a < b: 0
a >= b: 1
a <= b: 0

Process finished with exit code 0
```

Exercise 2.2: Calculate and display area of a rectangle with user-provided dimensions.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show their purposes)

Activity 2.2: Calculate and display the area of a rectangle with user-provided dimensions

LAB 3: Control Structures and Conditional Statements

Objectives:

1. Understand how control structures can make programs more efficient and dynamic.
2. Familiarity with Conditional Statements:
    - Introduce the if statement to execute code blocks conditionally.
    - Understand the if-else construct to handle binary decisions.
    - Learn the if-else if-else ladder for handling multiple conditions.

Experiment 3.1: Implement conditional statements (if, else if, else)

if Statement

- Write a simple C program to demonstrate the if statement.

Algorithm:

1. Start
2. Declare an integer variable named number.
3. Output a message requesting the user to enter a number.
4. Get the user's input and store it in the number variable using scanf.
5. Use an if statement to check whether number > 0.
6. If true, output: "The number [number] is positive."
7. End

Code:

```c
#include <stdio.h>

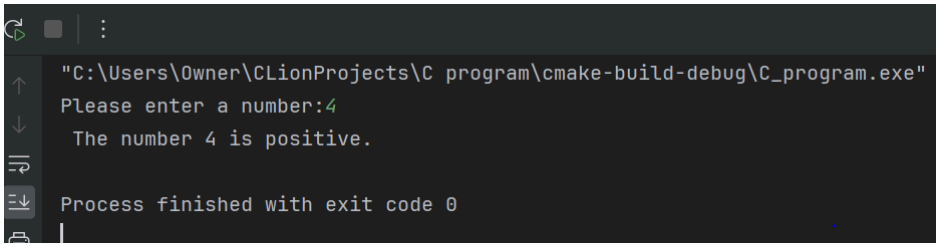
int main() {
    int number;

    // Get input from user
    printf("Please enter a number: ");
    scanf("%d", &number);

    // Check if the number is positive using if statement
    if(number > 0) {
        printf("The number %d is positive.\n", number);
    }

    return 0;
}
```

Output:



if-else Statement

- Write a simple C program to demonstrate if-else statement

Algorithm:

1. Start
2. Declare an integer variable named number.
3. Output a message requesting the user to enter a number.
4. Read the user's input and store it in the number variable using scanf.
5. Use an if-else statement to evaluate the number:
6. If number > 0, output: "The number [number] is positive."
7. Else if number < 0, output: "The number [number] is negative."
8. Else, output: "The number is zero."
9. End

Code:

```c
#include <stdio.h>

int main() {
    int number;

    // Get input from user
    printf( format: "Please enter a number: ");
    scanf( format: "%d", &number);

    // Check if the number is positive, negative, or zero using if-else statement
    if(number > 0) {
        printf( format: "The number %d is positive.\n", number);
    } else if(number < 0) {
        printf( format: "The number %d is negative.\n", number);
    } else {
        printf( format: "The number is zero.\n");
    }

    return 0;
}
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\cmake-build-debug\C_program.exe"
Please enter a number:-2
 The number -2 is negative.

Process finished with exit code 0
```

if-else Ladder

- Write a C program to check the grade of a student and print the corresponding grade letter (A, B, C, D, or F) based on the marks entered.

Algorithm:

1. Start
2. Declare an integer variable named marks.
3. Output a message requesting the user to enter their marks.

4. Read the user's input and store it in the marks variable using scanf.
5. Use an if-else-if ladder to evaluate the marks and determine the grade:
6. If marks is between 90 and 100, output: "Your grade is A."
7. Else if marks is between 80 and 89, output: "Your grade is B."
8. Else if marks is between 70 and 79, output: "Your grade is C."
9. Else if marks is between 60 and 69, output: "Your grade is D."
10. Else if marks is between 0 and 59, output: "Your grade is F."
11. Else, output: "Invalid marks entered."
12. End

Code:

```c
#include <stdio.h>
int main() {
    int marks;
    // Get input from user
    printf( format: "Please enter your marks: ");
    scanf( format: "%d", &marks);
    // Check the grade using if-else-if ladder
    if(marks >= 90 && marks <= 100) {
        printf( format: "Your grade is A.\n");
    } else if(marks >= 80 && marks < 90) {
        printf( format: "Your grade is B.\n");
    } else if(marks >= 70 && marks < 80) {
        printf( format: "Your grade is C.\n");
    } else if(marks >= 60 && marks < 70) {
        printf( format: "Your grade is D.\n");
    } else if(marks >= 0 && marks < 60) {
        printf( format: "Your grade is F.\n");
    } else {
        printf( format: "Invalid marks entered.\n");
    }
    return 0;
}
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\cmake-build-debug\C_program.
Please enter your marks:50
 Your grade is F.

Process finished with exit code 0
```

Exercise 3.1: Create a program to determine whether a given number is even or odd

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show their purposes)

Activity: Create a program to determine whether a given number is even or odd

Experiment 3.2: Use logical operators for complex conditions.

Logical operators allow us to combine multiple conditions to create complex conditional expressions. In C programming, the main logical operators are && (logical AND), II (logical OR), and ! (logical NOT).

Let's delve into examples for each of these logical operators:

Logical AND (&&)

- Write a C program to demonstrate logical AND operator

Algorithm:

1. Start
2. Declare two integer variables, num1 and num2.
3. Output a message requesting the user to enter two numbers.
4. Use scanf to read the two numbers entered by the user and store them in num1 and num2 respectively.

5. Use an if statement with the logical AND operator (&&) to check whether both num1 and num2 are positive:
6. If num1 > 0 && num2 > 0, output: "Both numbers are positive."
7. Else, output: "At least one number is not positive."
8. End

Code:

```c
#include <stdio.h>

int main() {
    int num1, num2;

    // Get input from the user
    printf( format: "Enter two numbers: ");
    scanf( format: "%d%d", &num1, &num2);

    // Check if both numbers are positive using logical AND operator
    if(num1 > 0 && num2 > 0) {
        printf( format: "Both numbers are positive.\n");
    } else {
        printf( format: "At least one number is not positive.\n");
    }

    return 0;
}
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\cmake-build-debu
Enter two numbers:4 5
 Both numbers are positive.

Process finished with exit code 0
```

In this program,

- If the user enters two positive numbers, e.g., 4 and 5, the output will be: "Both numbers are positive."
- If the user enters at least one non-positive number, e.g., -5 and 10, the output will be: "At least one number is not positive."

Logical OR (||)

- Write a Simple C program to demonstrate logical OR operator

Algorithm:

1. Start
2. Declare two integer variables, num1 and num2.
3. Print a message to the console asking the user to enter two numbers.
4. Use scanf to take two numbers as input from the user and assign them to num1 and num2.
5. Use an if statement with the logical OR operator (||) to check whether at least one of num1 and num2 is positive:
6. If num1 > 0 || num2 > 0, print: "At least one number is positive."
7. Else, print: "Neither number is positive."
8. End

code:

```c
#include <stdio.h>

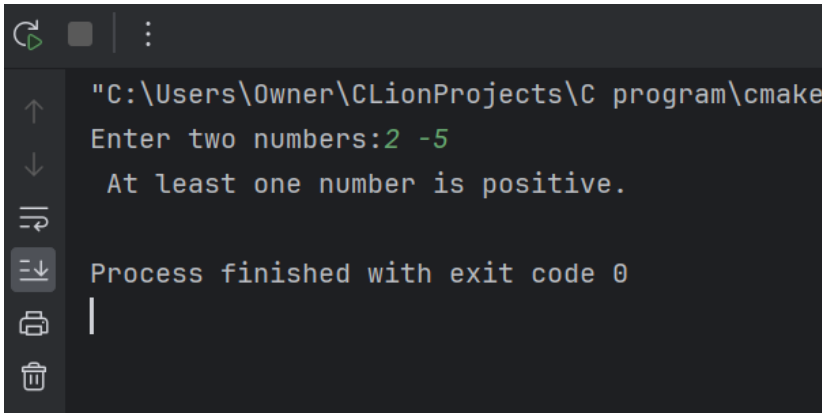
int main() {
    int num1, num2;

    // Get input from the user
    printf("Enter two numbers: ");
    scanf("%d%d", &num1, &num2);

    // Check if at least one number is positive using logical OR c
    if(num1 > 0 || num2 > 0) {
        printf("At least one number is positive.\n");
    } else {
        printf("Neither number is positive.\n");
    }

    return 0;
}
```

Output:



In this program:

- If the user inputs at least one positive number, e.g., 2 and -5, the output will be: "At least one number is positive."

- If the user inputs two non-positive numbers, e.g., -5 and -10, the output will be: "Neither number is positive."

Combining Logical Operators:

- Write a simple C program to demonstrate combining logical operators

Algorithm:

1. Start
2. Declare an integer variable, num.
3. Print a message asking the user to enter a number.
4. Use scanf to take an integer input from the user and store it in num.
5. Use an if statement with combined logical operators to check the condition:
6. If num == -10 OR (num >= 10 AND num <= 20), print: "The number is either -10 or in the range 10 to 20."
7. Else, print: "The number is neither -10 nor in the range 10 to 20."
8. End

Code:

```c
#include <stdio.h>

int main() {
    int num;

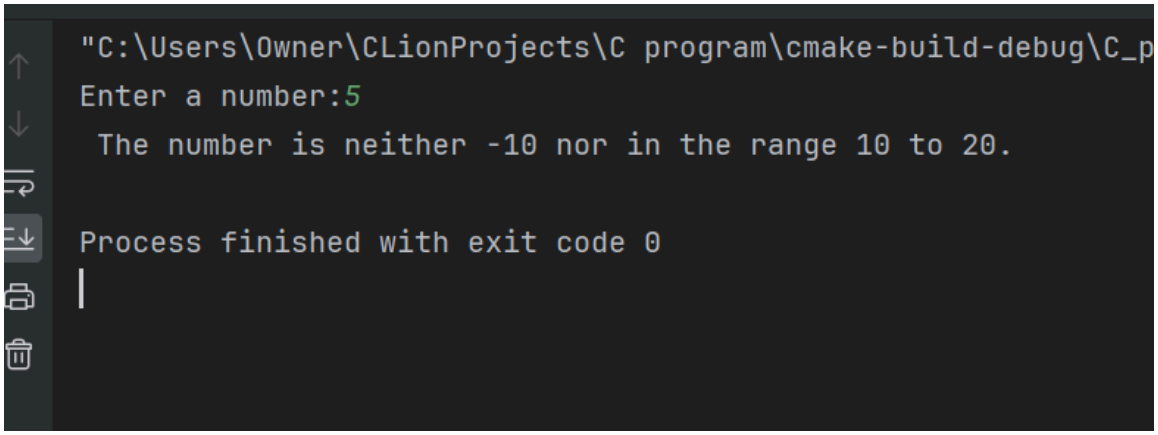
    // Get input from the user
    printf( format: "Enter a number: ");
    scanf( format: "%d", &num);

    // Check if the number is either equal to -10 or in the range 10 to 20 using combined logical operators
    if((num == -10) || (num >= 10 && num <= 20)) {
        printf( format: "The number is either -10 or in the range 10 to 20.\n");
    } else {
        printf( format: "The number is neither -10 nor in the range 10 to 20.\n");
    }

    return 0;
}

```

Output:



Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Exercise 3.2: Develop a calculator program that performs basic operations (addition, subtraction, multiplication, division) based on user input.

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs (Name of the variables should show their purposes)

LAB 4: Loops and Iteration

Objectives:

1. Learn the importance of loops in automating repetitive tasks and reducing code redundancy.
2. Get introduced to different types of loops available in C: for, while, and do-while.
3. Learn about the break statement to exit a loop prematurely.
4. Understand the continue statement to skip the current iteration and move to the next one.

Experiment 4.1: Implement for loops, while loops and do-while loops for repetitive tasks

- Write a simple C program that implements a for loop to print numbers from 1 to 10.

Algorithm:

1. Start.
2. Initialization: Set i to 1.
3. Loop Condition:
    - Check if i is less than or equal to 10.
    - If true, proceed to step 4.
    - If false, end the loop and proceed to step 5.
4. Loop Body:
    - Print the current value of i.
    - Increment the value of i by 1.
    - Return to step 3 to evaluate the loop condition again.
5. End.

Code:

```c
#include <stdio.h>

int main() {
    int i;

    // Use a for loop to print numbers from 1 to 10
    for (i = 1; i <= 10; i++) {
        printf("%d\n", i);
    }

    return 0;
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

In the program:

- The for loop starts with the initialization i = 1.
- It continues to iterate while the condition i < = 10 is true.
- After each iteration, the value of i is incremented by 1 (i++).
- Inside the loop body, the current value of i is printed

a. Write a simple C program that implements a while loop to print numbers from 1 to 10.

Algorithm:

1. Start.
2. Initialization: Set i to 1.
3. Loop Condition:
    - Check if i is less than or equal to 10.
    - If true, proceed to step 4.
    - If false, end the loop and proceed to step 5.
4. Loop Body:
    - Print the current value of i.
    - Increment the value of i by 1.
    - Return to step 3 to evaluate the loop condition again.
5. End.

Code:

```c
#include <stdio.h>

int main() {
    int i = 1;  // Initialization

    // Use a while loop to print numbers from 1 to 10
    while (i <= 10) {
        printf("%d\n", i);
        i++;  // Increment
    }

    return 0;
}
```

Output:



In the program:

- The for loop starts with the initialization i = 1.
- It continues to iterate while the condition i < = 10 is true.
- After each iteration, the value of i is incremented by 1 (i++).
- Inside the loop body, the current value of i is printed.

a. Write a simple C program that prints numbers from 1 to 10 using a do-while loop

Algorithm:

1. Start.
2. Initialization: Set i to 1.
3. Execute Loop Body:
    - Print the current value of i.
    - Increment the value of i by 1.
4. Loop Condition:
    - Check if i is less than or equal to 10.
    - If true, return to step 3.
    - If false, proceed to step 5.

5. End.

Code:

```c
#include <stdio.h>

int main() {
    int i = 1;  // Initialization

    // Use a do-while loop to print numbers from 1 to 10
    do {
        printf("%d\n", i);
        i++;  // Increment
    } while (i <= 10);  // Loop Condition

    return 0;
}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

In the program:

1. The value of i is initialized to 1.
2. The do-while loop guarantees that the loop body will be executed at least once before checking the condition.
3. After executing the loop body (printing the value of i and incrementing i), the loop checks the condition (i < = 10).

- If the condition is true, it re-enters the loop body.
- If the condition becomes false (when i is greater than 10), the loop exits

Exercise 4.1: Write a program to find the factorial of a user-provided integer using a loop

Students are to code the programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show their purposes)

Experiment 4.2: Utilize loop control statements (break and continue).

- Write a simple C program that prints numbers from 1 to 4 using a for loop and breaks out of the loop when the number is 5.

Algorithm:

1. Start.
2. Initialization: Set i to 1.
3. Loop Execution:
    - If i is equal to 5, proceed to step 4.
    - Print the current value of i.
    - Increment the value of i by 1.
    - Check if i is less than or equal to 10.
        - If true, return to the beginning of step 3.

- If false, proceed to step 5.

4. Break:
    - Exit the loop prematurely.
5. Post-loop Execution:
    - Print "Loop ended."
6. End.

Code:

```c
#include <stdio.h>

int main() {
    int i;

    // Use a for loop to print numbers from 1 to 10
    for (i = 1; i <= 10; i++) {
        if (i == 5) {
            break;  // Exit the loop if i is 5
        }
        printf("%d\n", i);
    }
    printf("Loop ended.\n");

    return 0;
}
```

Output:

```
1
2
3
4
Loop ended.
```

In the program:

- The value of i starts from 1 and increments by 1 in each iteration.
- When i becomes 5, the if condition becomes true and the break statement is executed. This causes the program to exit the for loop.

- After the loop, the message "Loop ended." is printed.

a. Write a simple C program that prints numbers from 1 to 10, but skips printing the number 5 using the continue statement.

Algorithm:

1. Start.
2. Initialization: Set i to 1.
3. Loop Execution:
    a. If i is equal to 5:
        - Skip the current iteration.
        - Increment the value of i by 1.
        - Return to the beginning of step 3.
4. Print the current value of i.
5. Increment the value of i by 1.
6. Check if i is less than or equal to 10.
    a. If true, return to the beginning of step 3.
    b. If false, proceed to step 4.
7. Post-loop Execution:
    a. Print "Loop ended."
8. End.

Code:

```c
#include <stdio.h>

int main() {
    int i;

    // Use a for loop to print numbers from 1 to 10
    for (i = 1; i <= 10; i++) {
        if (i == 5) {
            continue;  // Skip the current iteration if i is 5
        }
        printf("%d\n", i);
    }
    printf("Loop ended.\n");

    return 0;
}
```

Output:

```vbnet
1
2
3
4
6
7
8
9
10
Loop ended.
```

In the program:

- The value of i starts from 1 and increments by 1 in each iteration.
- When i is 5, the if condition becomes true and the continue statement is executed. This causes the program to skip the rest of the loop body for that iteration (i.e., the print statement) and jump to the next iteration.

- After the loop, the message "Loop ended." is printed.

Exercise 4.2: Implement a guessing game where the user has to guess a randomly generated number within a specified range.

Students are to code programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show their purposes)

LAB 5: Arrays and Strings

Objectives:

- Understand how to declare arrays with specific data types.
- Master the techniques to access, modify, and traverse array elements

Experiment 5.1: A simple C program that demonstrates how to declare, initialize, and work with a one-dimensional array.

Algorithm:

1. Start.
2. Array Declaration and Initialization:
    - Declare an integer array arr with a size of 5.
    - Initialize the array with the values {10, 20, 30, 40, 50}.
3. Print Original Array Elements:
    - For i from 0 to 4 (inclusive):
        - Print the value of the element at index i of the array arr.
4. Modify an Element in the Array:
    - Set the element at index 2 of the array arr to the value 100.
5. Print Modified Array Elements:
    - Print a newline and the message "Modified Array:".
    - For i from 0 to 4 (inclusive):
        - Print the value of the element at index i of the modified array arr.
6. End.

Code:

```c
#include <stdio.h>

int main() {
    // Declare and initialize an array of integers
    int arr[5] = {10, 20, 30, 40, 50};

    // Print the elements of the array
    for (int i = 0; i < 5; i++) {
        printf("Element at index %d: %d\n", i, arr[i]);
    }

    // Modify an element in the array
    arr[2] = 100;

    // Print the elements of the modified array
    printf("\nModified Array:\n");
    for (int i = 0; i < 5; i++) {
        printf("Element at index %d: %d\n", i, arr[i]);
    }
```

Output:

```
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
Element at index 4: 50

Modified Array:
Element at index 0: 10
Element at index 1: 20
Element at index 2: 100
Element at index 3: 40
Element at index 4: 50
```

In this program:

- An integer array arr with a size of 5 is declared and initialized.
- A for loop is used to print the elements of the array.
- The element at index 2 of the array is then modified.

- Another for loop is used to print the elements of the modified array.

Exercise 5.1: Create a program to find the sum and average of elements in an array accepting input from the user.

Students are to code the programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show their purposes)

Experiment 5.2: Perform array manipulation and traversal

- Write a simple C program to sum the element of an array accepting input from the user.

Algorithm:

1. Start.
2. Declare integer variables n, i, and sum. Initialize sum to 0.
3. Print "Enter the number of elements in the array:" to the console.
4. Read the value for n from the user.
5. Declare an integer array arr of size n.
6. For i from 0 to n-1 (inclusive):
    - Prompt the user with "Element i+1:".
    - Read the value for arr[i] from the user.
7. For i from 0 to n-1 (inclusive):
    - Add the value of arr[i] to sum.

8. Print "Sum of array elements: " followed by the value of sum.
9. End.

Code:

```c
#include <stdio.h>
int main() {
    int n, i;
    int sum = 0;
    // Ask user for number of elements
    printf( format: "Enter the number of elements in the array: ");
    scanf( format: "%d", &n);

    int arr[n];
    // Accept array elements from the user
    printf( format: "Enter %d elements:\n", n);
    for(i = 0; i < n; i++) {
        printf( format: "Element %d: ", i + 1);
        scanf( format: "%d", &arr[i]);
    }
    // Compute the sum of array elements
    for(i = 0; i < n; i++) {
        sum += arr[i];
    }
    printf( format: "Sum of array elements: %d\n", sum);

    return 0;
}
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\cmake-build-debug\C_program.exe"
Enter the number of elements in the array:3
 Enter 3 elements:
Element 1:5
 Element 2:7
 Element 3:8
 Sum of array elements: 20

Process finished with exit code 0
```

In this program:

1. **Variable Declaration:**
   - `int n, i;` : Here, `n` is used to store the number of elements in the array, and `i` is used as a loop counter.
   - `int sum = 0;`: This initializes the variable `sum` to `0`. It will be used to accumulate the sum of the array elements.

2. **Accepting the Array Size:**
    - The user is prompted to enter the number of elements they want in the array using printf.
    - The entered value is stored in the variable `n` using `scanf`.
3. **Array Declaration:**
    - `int arr[n];`: This declares an array of integers named `arr` with a size of `n`.
4. **Input Array Elements:**
    - A `for` loop is used to iterate `n` times. In each iteration, the program prompts the user to enter an element, and that element is stored in the respective position of the array using `scanf`.
5. **Compute the Sum:**
    - Another `for` loop iterates through the array, and each element of the array is added to the `sum`.
6. **Print the Result:**
    - Finally, the computed sum is printed to the console.

a. Write a simple C program that demonstrates traversal of a one-dimensional array.

Quick note: Traversal is the process of accessing each element in a data structure, such as an array, once.

Algorithm:

1. Start.
2. Declare integer variables n and i.

3. Print "Enter the number of elements in the array:" to the console.
4. Read the value for n from the user.
5. Declare an integer array arr of size n.
6. For i from 0 to n-1 (inclusive):
    - Prompt the user with "Element i+1:".
    - Read the value for arr[i] from the user.
7. Print "Traversing the array:" to the console.
8. For i from 0 to n-1 (inclusive):
    - Print "Element at index i: " followed by the value of arr[i].
9. End.

Code:

```c
#include <stdio.h>
int main() {
    int n, i;

    // Ask user for number of elements
    printf( format: "Enter the number of elements in the array: ");
    scanf( format: "%d", &n);

    int arr[n];
    // Accept array elements from the user
    printf( format: "Enter %d elements:\n", n);
    for(i = 0; i < n; i++) {
        printf( format: "Element %d: ", i + 1);
        scanf( format: "%d", &arr[i]);
    }

    // Traversal of the array elements
    printf( format: "\nTraversing the array:\n");
    for(i = 0; i < n; i++) {
        printf( format: "Element at index %d: %d\n", i, arr[i]);
    }
    return 0;
}
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\cmake-build-debug\C_program.exe"
Enter the number of elements in the array:4
 Enter 4 elements:
Element 1:11
 Element 2:5
 Element 3:34
 Element 4:7

Traversing the array:
Element at index 0: 11
Element at index 1: 5
Element at index 2: 34
Element at index 3: 7

Process finished with exit code 0
```

In this program:

1. **Variable Declaration:**
    - int n, i;: Here, n is used to store the number of elements in the array, and i is used as a loop counter.
2. **Accepting the Array Size:**
    - The user is prompted to enter the number of elements they want in the array using printf.
    - The entered value is stored in the variable n using scanf.
3. **Array Declaration:**
    - int arr[n];: This declares an array of integers named arr with a size of n.
4. **Input Array Elements:**
    - A for loop is used to iterate n times. In each iteration, the program prompts the user to enter an element, and that element is stored in the respective position of the array using scanf.

5. Traversal:
    - A for loop is used to traverse (or iterate through) each element of the array. During traversal, the program prints out the index and value of each element.
6. End:
    - The program completes its execution and returns 0.

Exercise 5.2: Implement a program that finds the largest and smallest values in an array accepting input from the user and traverses the array.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show their purposes)

Activity: Implement a program that finds the largest and smallest values in an array accepting input from the user and traverses the array.

LAB 6: Functions and Modular Programming

Objectives:

- Understand the syntax and structure of defining a function.
- Comprehend the concept of passing arguments to functions.
- Understand how functions return values using the return statement

Experiment 6.1: Write a simple C program that defines a function to add two numbers

Algorithm: Start.

1. **Function Declaration:**
    - Declare a function named add that takes in two integer parameters and returns an integer.

Begin main execution

2. **Variable Declaration:**
    - Declare three integer variables: x, y, and result.
3. **Input:**
    - Display the message "Enter two numbers:" to the user.
    - Read two integer values from the user and store them in x and y.
4. **Function Call:**
    - Call the add function with x and y as arguments.
    - Store the returned value of the add function in the result variable.
5. **Output:**
    - Display the message "The sum of x and y is: result".

End main execution

6. **Function Definition (add):**
    - Take two integer parameters, a and b.
    - Return the sum of a and b.
7. **End.**

Code:

```c
#include <stdio.h>
// Function declaration (prototype)
int add(int a, int b);

int main() {
    int x, y, result;

    // Accepting two numbers from the user
    printf( format: "Enter two numbers:\n");
    scanf( format: "%d%d", &x, &y);

    // Calling the add function and storing its return value
    result = add( a: x,  b: y);

    // Displaying the result
    printf( format: "The sum of %d and %d is: %d\n", x, y, result);

    return 0;
}
// Function definition
int add(int a, int b) {
    return a + b;
}
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\cmake-build-debug\C_program.exe"
Enter two numbers:
3
8
The sum of 3 and 8 is: 11

Process finished with exit code 0
```

In this program:

- We first declare a function prototype for the `add` function.
- Inside `main()`, we get two numbers from the user.
- We then call the `add` function with these two numbers as arguments.
- The `add` function calculates their sum and returns it.
- The returned sum is then printed to the console from the `main()` function.

Exercise 6.1: Develop a program that calculates the area of various geometric shapes (circle, rectangle, triangle) using separate functions

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show their purposes)

Activity: Develop a program that calculates the area of various geometric shapes (circle, rectangle, triangle) using separate functions.

Experiment 6.2: Understand function prototypes and header files.

Note: In C programming, a function prototype (also known as a function declaration) provides the compiler with information about the function's name, return type, and parameters. A header file, typically with a .h extension, is a file where you can declare function prototypes, global variables, macro definitions, and other declarations that can be shared across multiple source files.

Algorithm:

1. Start
2. Create the header file named circle.h
3. Declare the function area_of_circle in circle.h
4. Create a main program named main.c
5. Define the function area_of_circle in the main.c file

6. Request data and convert the function

Write a c program to calculate the area of a circle.

- First, create a header file named `circle.h`

```c
// circle.h

#ifndef CIRCLE_H
#define CIRCLE_H

// Function prototype
double area_of_circle(double radius);

#endif // CIRCLE_H
```

- Now, create the C program named main.c:

```c
#include <stdio.h>
#include "circle.h" // Include the header file

// Define PI
#define PI 3.14159

// Function definition
double area_of_circle(double radius) {
    return PI * radius * radius;
}

int main() {
    double r;

    printf( format: "Enter the radius of the circle: ");
    scanf( format: "%lf", &r);

    printf( format: "Area of the circle with radius %.2lf: %.2lf\n", r, area_of_circle( radius: r));

    return 0;
}
```

Explanation:

1. The header file `circle.h` contains the prototype of the function `area_of_circle()`. This tells the compiler what the function looks like.

2. The C program `main.c` includes this header file using the `#include` directive. This way, the compiler knows about the function prototype when compiling the program.
3. The function `area_of_circle()` is then defined in `main.c`, and this definition provides the actual implementation of the function.
4. The `main()` function prompts the user to input a radius, calculates the area using the `area_of_circle()` function, and then displays the result.

Exercise 6.2: Create a library of utility functions (e.g., factorial, power) and use them in multiple programs.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show their purposes)

LAB 7: Pointers and Dynamic Memory Allocation

Objectives:

- Learn about pointer variables, their declaration, and the dereferencing operator.
- Get acquainted with memory allocation functions like malloc() and deallocation function free().

Experiment 7.1: Write a C program to create, initialize, assign and access a pointer variable

Algorithm:

1. Start.
2. Declare an integer variable named num.
3. Declare an integer pointer named pNum.
4. Assign the address of the num variable to the pointer pNum.
5. Assign the value 100 to the variable num.
6. Display the message "Using variable num:".
7. Print the value of num using the variable itself.
8. Print the address of num using the address-of operator &.
9. Display the message "Using pointer variable:".
10. Print the value of num by dereferencing the pointer pNum using the dereference operator *.
11. Print the address of num using the pointer pNum.
12. End.

Code:

```c
/*C program to create, initialize, assign and access a pointer variable.*/

#include <stdio.h>

int main()
{
    int num;    /*declaration of integer variable*/
    int *pNum;  /*declaration of integer pointer*/

    pNum=& num; /*assigning address of num*/
    num=100;    /*assigning 100 to variable num*/

    //access value and address using variable num
    printf( format: "Using variable num:\n");
    printf( format: "value of num: %d\naddress of num: %u\n",num,&num);
    //access value and address using pointer variable num
    printf( format: "Using pointer variable:\n");
    printf( format: "value of num: %d\naddress of num: %u\n",*pNum,pNum);

    return 0;
}
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\cmake-build-debug\C_program.exe"
Using variable num:
value of num: 100
address of num: 3632265092
Using pointer variable:
value of num: 100
address of num: 3632265092

Process finished with exit code 0
```

In the output:

- The value of num will always be 100.
- The addresses shown for &num and pNum will be the same, but the specific address will vary depending on the environment and each run of the program.

Exercise 7.1: Write a C program to Swap the values of two variables using pointers.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs (Name of the variables should show their purposes)

Experiment 7.2: Write a C program to create memory for int, char and float variable at run time (using malloc and free)

Algorithm:

1. Start.
2. Declare three pointers:
    - iVar for an integer.
    - cVar for a character.
    - fVar for a float.
3. Memory Allocation:
    - Allocate memory dynamically for an integer using malloc and store its address in the iVar pointer.
    - Allocate memory dynamically for a character using malloc and store its address in the cVar pointer.
    - Allocate memory dynamically for a float using malloc and store its address in the fVar pointer.

4. Prompt the user with the message "Enter integer value: ".
5. Read an integer value from the user and store it in the memory location pointed to by iVar.
6. Prompt the user with the message "Enter character value: ".
7. Read a character value from the user (using a space before %c format specifier to skip any previous newline or whitespace) and store it in the memory location pointed to by cVar.
8. Prompt the user with the message "Enter float value: ".
9. Read a float value from the user and store it in the memory location pointed to by fVar.
10. Print the values entered by the user by dereferencing the pointers: *iVar, *cVar, and *fVar.
11. Memory Deallocation:
    - Release the dynamically allocated memory for the integer using free.
    - Release the dynamically allocated memory for the character using free.
    - Release the dynamically allocated memory for the float using free.
12. End.

Code:

```c
/*C program to create memory for int,
char and float variable at run time.*/

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *iVar;
    char *cVar;
    float *fVar;

    /*allocating memory dynamically*/
    iVar=(int*)malloc( Size: 1*sizeof(int));
    cVar=(char*)malloc( Size: 1*sizeof(char));
    fVar=(float*)malloc( Size: 1*sizeof(float));

    printf( format: "Enter integer value: ");
    scanf( format: "%d",iVar);

    printf( format: "Enter character value: ");
    scanf( format: " %c",cVar);

    printf( format: "Enter float value: ");
    scanf( format: "%f",fVar);

    printf( format: "Inputted value are: %d, %c, %.2f\n",*iVar,*cVar,*fVar);

    /*free allocated memory*/
    free( Memory: iVar);
    free( Memory: cVar);
    free( Memory: fVar);

    return 0;
}
```

Output:

```
"C:\Users\Owner\CLionProjects\C program\cmake-build-debug\C_program.exe"
Enter integer value:2
 Enter character value:C
 Enter float value:0.1
 Inputted value are: 2, C, 0.10

Process finished with exit code 0
```

In this example:

- The user entered 2 for the integer value.
- The user entered C for the character value.
- The user entered 0.1 for the float value.
- The program then displays the values entered by the user.

Exercise 7.2: Implement a program that dynamically allocates memory for an array based on user input and displays the elements in reverse order.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs(Name of the variables should show their purposes)

LAB 8: File Handling and Structures

Objectives:

- Understand the process of reading from and writing to files.
- Learn how to define, declare, and initialize structures.

Experiment 8.1: Perform basic file input and output operations

- Write a C program to create, open, write a message to a file and close a file.

Algorithm:

1. Start.
2. Declare a pointer of type `FILE` named `filePtr`.
3. Use the `fopen()` function to open (or create if it doesn't exist) a file named "sample.txt" in write mode ("w") and assign the returned value to filePtr.
4. Check if the file was opened successfully:
    - If `filePtr` is NULL, then:
        - Display an error message: "Error! Unable to open the file."
        - Terminate the program with an error code.
5. Use the `fprintf()` function to write the message "Hello, this is a message written to the file!" to the file pointed by `filePtr`.
6. Close the file using the `fclose()` function.
7. Display a success message: "Message written to the file successfully."
8. End.

Code:

```c
#include <stdio.h>
int main() {
    FILE *filePtr;   // Pointer to file

    // Open (or create) a file named "sample.txt" for writing
    filePtr = fopen( Filename: "sample.txt",  Mode: "w");

    // Check if the file was opened successfully
    if (filePtr == NULL) {
        printf( format: "Error! Unable to open the file.");
        return 1;  // Return with an error code
    }

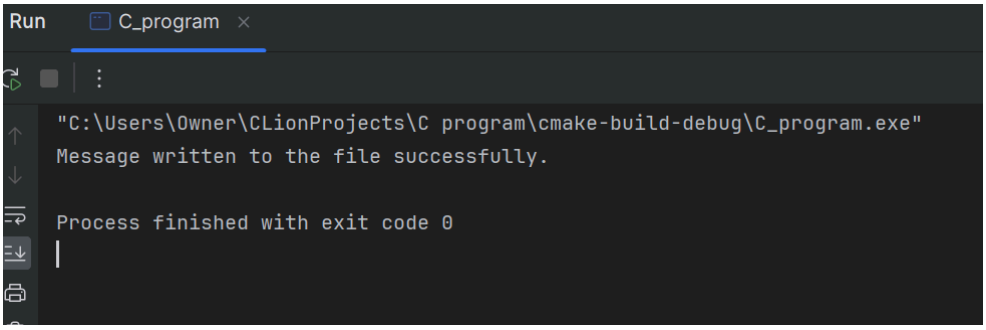
    // Write a message to the file
    fprintf( stream: filePtr,  format: "Hello, this is a message written to the file!");

    // Close the file
    fclose( File: filePtr);

    printf( format: "Message written to the file successfully.\n");

    return 0;
}
```

Output:



Explanation:

- The fopen() function is used to open (or create) a file named "sample.txt" in write mode ("w").
- If the file cannot be opened (e.g., due to permissions), an error message is printed.
- The fprintf() function is used to write a message to the file.

- The fclose() function is used to close the file after writing to ensure all buffers are flushed and no data is lost.
- A success message is printed to the console to notify the user.

Exercise 8.1: Create a program that reads data from a given text file, processes it to store the values in the text file into an array, find the average of the number and writes the results to another file.

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs (Name of the variables should show their purposes)

Experiment 8.2: Define and use structures to organize data.

- Write a C program that demonstrates the use of a structure to represent an employer's information accepting the input from the user.

Algorithm:

1. Start.
2. **Include Necessary Libraries:**
    - Include the standard input-output library ‹stdio.h›.

Structure Declaration:

3. Define a structure named employee with the following members:
    - `name` (an array of 30 characters).
    - `empId` (of integer type).

- `salary` (of float type).

Begin Main Execution:

4. Declare and Initialize a Structure Variable:
    - Declare a variable emp of type employee.
5. **Initialize the variable with the following values:**
    - `name` as "Mike".
    - `empId` as 1120.
    - `salary` as 76909.00f.
6. **Print the Employee's Information:**
    - Display the employee's name using the format "Name: %s" and access it with `emp.name`.
    - Display the employee's ID using the format "Id: %d" and access it with `emp.empId`.
    - Display the employee's salary using the format "Salary: %f" and access it with `emp.salary`.
7. Terminate the program with a return value of 0.

End Main Execution.

8. End.

Code:

```c
#include <stdio.h>

/*structure declaration*/
struct employee {
    char name[30];
    int empId;
    float salary;
};

int main()
{
    /*declare and initialization of
    structure variable*/
    struct employee emp = { .name: "Mike", .empId: 1120, .salary: 76909.00f };

    printf( format: "\n Name: %s", emp.name);
    printf( format: "\n Id: %d", emp.empId);
    printf( format: "\n Salary: %f\n", emp.salary);

    return 0;
}
```

Output:



Explanation:

- The code defines a data structure named employee which represents an employee's details: their name, ID, and salary.
- In the main function, an instance of this structure, emp, is created and initialized with the values: name as "Mike", ID as 1120, and salary as 76909.00f.

- The program then prints these values to the console using the printf function. Specifically, it displays the employee's name, ID, and salary in separate lines.

Exercise 8.2: Design a structure to represent a student's information (name, roll number, marks) and perform operations on a list of students

Students are to code the above programs on their PC and submit on Code Grader to instructor/course coordinator

Instructions:

- Write a comment to make your programs readable.
- Use descriptive variables in your programs (Name of the variables should show their purposes)

LAB 9: Final Project (CA) - 2 weeks

Project Title: Student Record System in C programming.

Project Description:

You are tasked with creating a simple **Student Record System in C** that allows users to manage student information. The program should provide the following functionality:

1. Create a new C project in our preferred development environment (JetBrains Clion).
2. Write a program that displays a welcome message and allows the user to enter their name. The program should then greet the user with their name.
3. Implement data storage for student information, including name, roll number, and marks. Use appropriate data types for each field.
4. Add a feature that allows users to input a student's marks and then displays whether the student has passed or failed based on a passing threshold (e.g., marks above 40 are considered passing).
5. Implement a loop that allows the user to input information for multiple students. Continue to add students until the user decides to exit.
6. Create an array to store information for multiple students. You should be able to add, remove, and modify student records within this array.
7. Implement functions for adding, modifying, and displaying student records. These functions should take appropriate arguments and return relevant information.

8. Use dynamic memory allocation to allocate memory for the student records. Ensure you free the memory when it is no longer needed.
9. Implement the ability to save student records to a text file and load student records from a text file. The file should store student information like name, roll number, and marks.

Additional Features

- Allow users to search for a student by their roll number and display their information.
- Calculate and display the average marks for all students.
- Provide an option to sort the student records based on marks in ascending or descending order.

Project Submission

Submit your C program along with a brief documentation explaining how to use it. Make sure your code is well-organized, and you have used appropriate comments and function headers.

Notes

1. A very good manual
2. There should be a cover page
3. Modify as suggested
4. Provide enough spaces between sentences and screenshots

Edit and revert to me for final approval